D H A R

Volume II — Engineering Roadmap

STAGE-DELTA

“Gearing Dhar — and Kun: declare what should exist, and it does.”

Document	Stage-delta — Gearing Dhar
Version	2.1
Date	September 7, 2026
Status	Ratified
Part of	The Canon of Dhar, Volume II
Depends on	Spec0 §0 (naming), Spec1 §11 (Collections), §18 (foreign)
Precedes	Stage-epsilon — Deploying Dhar

1. Mandate

Give Dhar a standard library, a named and fully specified package manager, and a developer toolchain good enough that writing real software in it does not require reinventing basic infrastructure — and give the collection types Spec1 §11 already fixed their real implementations.

2. Standard Library

Module	Covers
std.io	Console and file input/output, built on gen/sys (Spec1 §20)
std.mem	Arena helpers and manual-escape-hatch allocation utilities
std.collections	list, set, dict, tuple support code, plus Stack/Queue/Deque (Spec1 §13.7)
std.fmt	String formatting, building on f-string interpolation (Spec1 §2.4)
std.net	Networking
std.fs	Filesystem access
std.crypto	Cryptographic primitives
std.math	Numeric functions beyond the core operators (Spec1 §6)
std.concurrent	Higher-level concurrency utilities built on spark/draw (Spec1 §15)

3. Kun — The Package Manager

3.1 Why “Kun”

Named for the Arabic *kun*, “be” — the imperative in the Quranic *kun fa-yakun*, “be, and it is” (Spec0 §0). A package manager's core promise is the same shape at a vastly smaller scale: declare what a project needs, and it comes into being — resolved, fetched, verified, ready — without being assembled by hand. The name is a mandate as much as a label: everything in this section exists to make that declaration-to-existence promise actually true, not just poetic.

3.2 Project manifest

Every Kun-managed project declares, in one manifest file: project name and semantic version; dependencies, each with a version constraint; the minimum Dhar language version required (Spec1's own version, per its revision log); and build target(s) (Spec0 §4's backend list).

3.3 Dependency resolution & lockfiles

1. Kun must resolve a full dependency graph deterministically: the same manifest must always produce the same resolution. This is not a quality goal, it is a direct precondition for Stage-epsilon's reproducible-builds requirement — a build cannot be reproducible if its own dependency resolution is not.

2. Resolution produces a lockfile recording every resolved version and its content hash. A second kun install against an unchanged manifest must reuse the lockfile exactly, never silently re-resolve to different versions, and must do so entirely from local cache with no network round-trip required — a design requirement, not an optimization, since Casting Dhar (Volume III) documents real, widespread connectivity constraints across Dhar's primary target geography that a network-dependent install step would fail against.

3. A version conflict that cannot be resolved must fail loudly, with the conflicting requirements shown, rather than silently selecting one and hoping.

3.4 Versioning

Kun enforces semantic versioning (MAJOR.MINOR.PATCH) for anything published to the public registry: a breaking change requires a MAJOR bump, checked where practical against the package's own declared bond (Spec1 §12) and mold field signatures, not merely taken on the publisher's word.

3.5 Workspaces

A single manifest may declare multiple local packages sharing one lockfile and one resolution — required for a project shaped like Dhar's own (compiler, standard library, and tooling as separate packages built together), and for any real application split into internal libraries rather than one monolith.

3.6 Registry & publishing

Kun **must** support a public registry: name reservation, and immutable published versions — once published, a version's content cannot change, only be yanked. Kun **must also** support pulling from a private, self-hosted registry as a first-class configuration, not an afterthought: an institution building critical infrastructure (Spec0 §5) cannot make its supply chain depend on a single externally controlled registry, which would quietly reintroduce the exact dependency Spec0 §2.1 exists to remove.

3.7 Security & provenance

- Every published package carries a content hash and a signature, verified on install.
- Kun must be able to emit an SBOM entry for every resolved dependency, feeding directly into Stage-epsilon's SBOM requirement — a release's software bill of materials is only as complete as its package manager's records.
- A yanked version must still be resolvable for existing lockfiles (so old builds keep building) but must not be selected for any new resolution.

4. Interop & the Foreign Ecosystem

Dhar's own ecosystem, distributed through Kun, is necessary but not sufficient. Real adoption requires that Dhar code can call into, and be called from, the existing ecosystems of C, C++, Rust, Java, and Python. Spec1 §18 specifies `foreign`, the single interop primitive everything below rests on: a Dhar declaration of an external, C-ABI-compatible function. Every language here is reachable through that one mechanism, though the directness of the path varies — and this document says so honestly rather than promising uniform ease that would not survive contact with reality.

Language	Path	Directness
C	foreign declares the function directly; C's ABI is exactly what foreign targets.	Direct
C++	A thin extern “C” wrapper compiled alongside the C++ library, then targeted by foreign — the same solution every language uses, since C++ has no single cross-compiler ABI to target directly.	Indirect, by necessity
Rust	A Rust function marked #[no_mangle] extern “C” exposes a stable C boundary — already common practice via -sys/-ffi crates — which foreign then targets.	Indirect, standard practice
Java	A JNI (Java Native Interface) bridge, the same mechanism every non-JVM language uses to reach the JVM; foreign targets the bridge's C side.	Indirect, requires a shim
Python	Two directions: Dhar calling Python embeds CPython via its C API (Python.h) through foreign; Python calling Dhar loads a Dhar shared library (built from expose-d functions) via ctypes/cffi, with no change needed on the Dhar side.	Indirect one way, near-direct the other

Kun's role in all five: fetching and building the wrapper/shim crates each indirect path needs is an ordinary dependency resolution, not a special case, and generating the JNI boilerplate automatically is a strong candidate for Stage-delta-and-later tooling work rather than a core-language feature.

The common thread is deliberate: one interop keyword, targeting the one interop layer every mature ecosystem already knows how to speak, directly or through a thin, well-established bridge. That is more auditable than five different interop mechanisms each claiming a seamlessness none of them fully deliver — and more honest about where the real friction actually is.

5. Developer Tooling

- `dhar build` — builds a multi-file, multi-module, multi-package (workspace-aware) project from one command.
- `dhar test` — discovers and runs tests without per-file configuration.
- `dhar fmt` — the canonical formatter, resolving any style question Spec1 leaves to convention.
- A documentation generator, producing reference docs from source comments.
- LSP support: diagnostics and go-to-definition at minimum.

6. Required Architecture

The standard library is written in Dhar itself, using Stage-gamma's compiler, and follows the same one-definition-per-file discipline as every earlier stage. Kun is written in Dhar as well, and is itself the first real test of the standard library's std.net and std.fs modules under load. Compiler intrinsics are permitted only where Spec1's memory model (§14) or performance genuinely requires them; everything else **must** be ordinary Dhar code, per the design stance Spec1 §13 already commits to.

7. Acceptance Criteria

1. Every collection type in Spec1 §13 has a working implementation matching its specified literal syntax exactly.

2. Kun resolves a real multi-package workspace deterministically, and a second install against an unchanged manifest reuses the lockfile without re-resolving.

3. Kun can publish to and install from both a public registry and a self-hosted private one, with no code path that assumes only the former exists.

4. At least one working interop example exists for each of C, C++, Rust, Java, and Python, per §4's table — not merely specified, demonstrated.

5. dhar build builds a real multi-file, multi-module project from a single command; dhar test discovers and runs that project's tests with zero per-file configuration.

6. LSP support is verified against at least one real editor integration.

8. Handoff to Stage-epsilon

Stage-epsilon assumes a working build system and a working, secured package manager already exist; it hardens what this stage builds — including consuming Kun's SBOM entries directly (Stage-epsilon §2) — rather than building either from nothing.

Stage-delta — Gearing Dhar · v2.0 · September 7, 2026 · Ratified. Revision log: v1.0 — first ratified edition. v2.0 — named and fully specified the package manager as Kun (§3: manifest, lockfiles, versioning, workspaces, registry, security/provenance); added §4, Interop & the Foreign Ecosystem, covering C, C++, Rust, Java, and Python via Spec1's foreign keyword. v2.1 — §3.3

now requires lockfile reuse entirely from local cache with no network round-trip, per Casting Dhar's connectivity findings.